Setup Guide

How to install and run the Booking Management System on any computer.

This guide gets the Booking System running on any computer (Windows, macOS, or Linux) from a clean start. Follow the steps in order. You only do steps 1–3 once.

What you need to install

Three free tools. Install each, then check the version to confirm it worked.

Tool	Why	Check it installed
Java JDK 17+	Runs the backend	java -version
Apache Maven 3.9+	Builds the project	mvn -version
PostgreSQL 15+	The database	psql --version

- **Java:** download from adoptium.net (Temurin 17 or newer).
- **Maven:** from maven.apache.org/download — or use your package manager (`brew install maven`, `sudo apt install maven`).
- **PostgreSQL:** from postgresql.org/download. During install, remember the password you set for the `postgres` user.

> **Note:** Apache JMeter (for the load test in step 6) is optional and only needed if you want to reproduce the concurrency demo.

Step 1 — Get the project

Unzip the project folder (or clone it) somewhere convenient, then open a terminal **inside the booking-system folder** (the one that contains `pom.xml`).

```
cd path/to/booking-system
ls # you should see pom.xml, src/, db/, jmeter/
```

Step 2 — Create the database

Create an empty database called `booking_db`. The application creates the tables itself on first launch, so you do not need to run any SQL by hand.

```
createdb -U postgres booking_db
# If 'createdb' is not found, use the SQL console instead:
# psql -U postgres
# CREATE DATABASE booking_db;
# \q
```

Step 3 — Tell the app your database password

Open `src/main/resources/application.properties` and set the username/password to match your PostgreSQL install. The defaults are user `postgres` / password `postgres`.

```
spring.datasource.url=jdbc:postgresql://localhost:5432/booking_db
spring.datasource.username=postgres
spring.datasource.password=YOUR_PASSWORD_HERE
```

> **Note:** Prefer not to edit the file? Set environment variables instead: DB_URL, DB_USER and DB_PASSWORD. The app reads those if present.

Step 4 — Run the application

From the project folder, start it with Maven. The first run downloads the libraries, so give it a minute.

```
mvn spring-boot:run
```

When you see a line like `Started BookingApplication in ... seconds`, it is ready. Sample users and resources are inserted automatically.

Step 5 — Open the interface

In your browser, go to:

```
http://localhost:8080
```

You should see the booking interface. Pick a resource, choose a date, and click an available slot to book it. Booking the same slot twice on a capacity-1 resource is rejected — that is the whole point of the project working.

Step 6 — (Optional) Prove it under load with JMeter

- Install Apache JMeter (jmeter.apache.org) and launch it.
- Open `jmeter/booking-stress-test.jmx`.
- Restart the app first (so the 14:00 demo slot is free), then press Run.
- Open **View Results Tree**: exactly one request returns `201 Created` and the other 49 return `409 Conflict`. Zero double-bookings.

Step 7 — (Optional) Add the database safety-net

For an extra guarantee on capacity-1 resources, apply the EXCLUDE constraint. This makes PostgreSQL itself reject overlaps even if the application code had a bug.

```
psql -U postgres -d booking_db -f db/hardening.sql
```

> **Note:** Do not apply this if you use capacity > 1 resources — see the comments inside hardening.sql for why.

Running the automated tests

The unit and integration tests need no database (they use an in-memory one). From the project folder:

```
mvn test
```

Common problems

Symptom	Fix
Port 8080 already in use	Stop the other program, or change server.port in application.properties.
Connection refused to database	PostgreSQL is not running, or the password in application.properties is wrong.
'mvn' not recognised	Maven is not installed or not on your PATH. Re-check step under 'What you need'.
UI says 'Could not reach the server'	The backend is not running. Start it with mvn spring-boot:run.